

기술개발 계획서

(빌딩 운영 연합 D/T 개발 및 서비스 고도화)

(주) 제이에이치솔루션

'25. 5. 28

〈 목 차 〉

1. 목 표

1.1. 목표 성능	01
1.2. 검증 방안	04

2. 추진 전략

2.1. 추진 계획	05
2.2. 추진 방법	12
2.3. 수행 일정	27

1. 목 표

빌딩 운영 연합 D/T 기술 개발 및 서비스 고도화

1. 개발 필요성

현재 수준 (Lv.3 모의 D/T)

- 설비 수동제어
- Asset, 3D 모델 수동 동기화
- 개별 시뮬레이션 수행
- 로컬 서버 운영



목표 수준 (Lv.4 연합 D/T)

- 시뮬레이션 결과 기반 설비 자동제어
- Asset, 3D 모델 자동 동기화
- 통합 시뮬레이션 수행
- 하이브리드(로컬, 클라우드) 운영

2. 개발 기술

연합 디지털 트윈 기술 개발

- 1 빌딩 설비 연동 기술
- 2 빌딩 설비 제어 기술
- 3 Asset 자동 동기화 기술
- 4 3D 모델 자동 동기화 기술
- 5 통합 시뮬레이션 기술

BEES 플랫폼 서비스 고도화

- 6 지능형 공간 관리 서비스 개발
- 7 Multi-Tenant 서비스 개발
- 8 보안 및 인증체계 구축
- 9 클라우드 서비스 확장
- 10 웹/모바일 인터페이스 확장

3. 고객 가치

연합 디지털 트윈 기술 개발

- 1 자동화된 설비 제어
- 2 실시간 자산 관리
- 3 디지털 트윈 정확성 향상
- 4 시뮬레이션 시스템 최적화
- 5 에너지 효율 향상

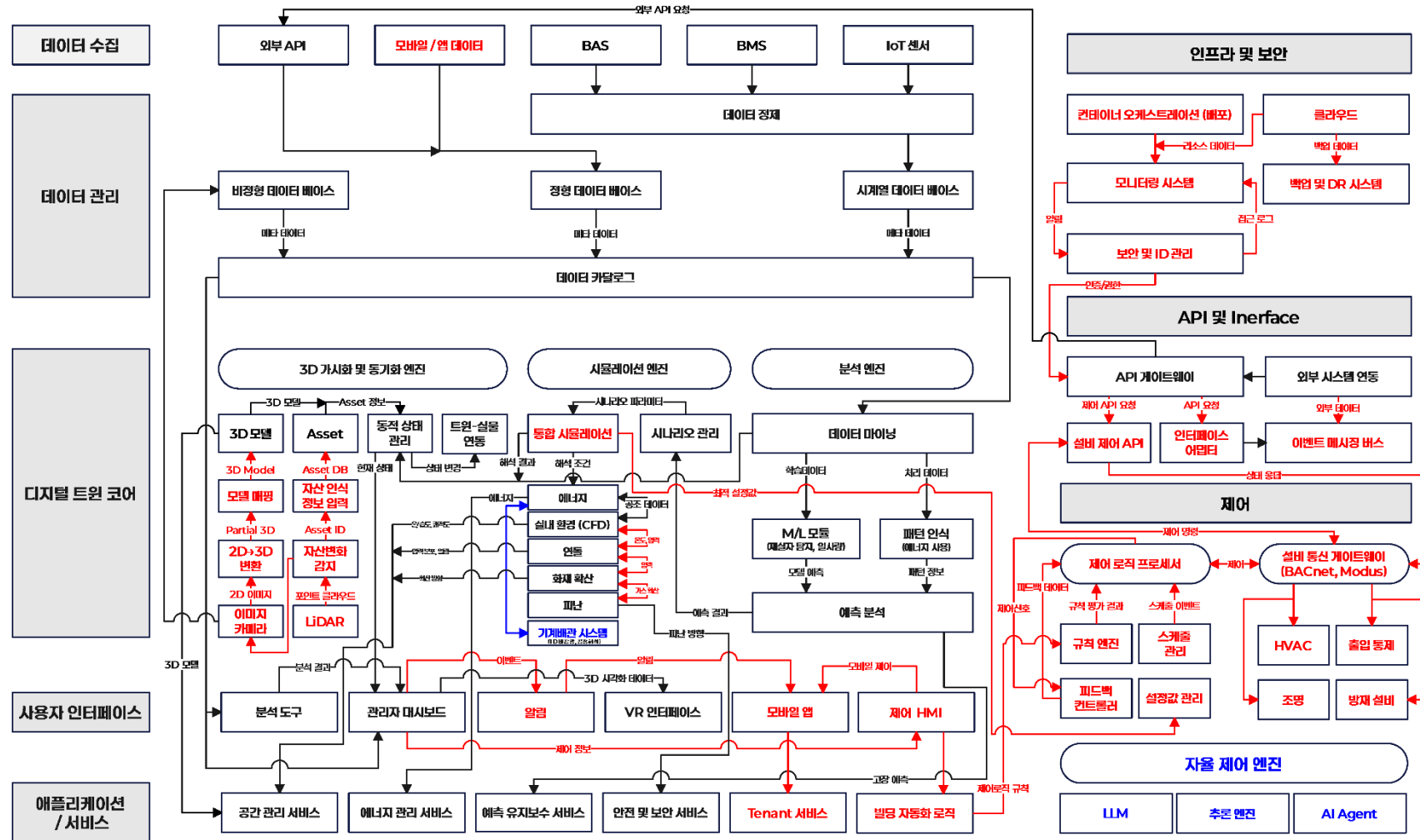
BEES 플랫폼 서비스 고도화

- 6 공간 활용 최적화
- 7 다중 건물 통합 관리
- 8 데이터 보안 강화
- 9 유연한 서비스 이용
- 10 접근성 확대

연합 디지털 트윈 기술과 서비스 고도화를 통해
Level 4 디지털 트윈 플랫폼을 구현하고 고객별 맞춤형 가치 제공

BEES System Schematic Diagram

개발 완료
금회 개발
개발 예정



* 금회 과제 완료 전, 자율제어 로직 구축 예정 (~'26.02)

1.1. 목표 성능

구분	항목	설명	목표 성능
빌딩 설비 연동/제어	실시간 데이터 업데이트	센서 및 설비 상태 데이터가 디지털 트윈 및 모니터링 대시보드에 반영되는 시간	3 초 이내 (최대 5 초 미만)
	제어 명령 응답 시간	제어 API 를 통한 명령 전송 및 실행 응답 시간	500ms 미만
	시스템 가용성	안정적인 데이터 수집율 및 제어 수행율	95% 이상
Asset 자동 동기화	데이터 처리 및 응답 속도	안정적으로 동작하기 위한 CRUD API 응답 시간	300ms 미만
	데이터 정확성 및 일관성	자산 정보(자재 고유 정보, 설치 위치, 수명, 교체 이력 등)의 정확성 및 실시간 업데이트 보장	95% 이상
3D 모델 자동 동기화	3D 모델링 정확도	생성된 3D 모델의 실제 치수 대비 오차율	±5% 이내
	처리 속도	사진 기반 3D 모델 생성 시간	건물 크기에 따라 30 분 이내 (일반 건물 기준)
통합 시뮬레이션	에너지 최적화 정확도	에너지 시뮬레이션의 예측 오차율	5% 미만
	실내 환경/연돌 분석데이터 고도화	실시간 IoT 센서 데이터(온도, 습도 등)를 활용한 실내 환경 및 연돌 분석 데이터 정합성 개선	±10% 이내
	화재/재난 대피 대응 가이드	화재 시나리오에 따른 세분화된 분석을 통한 이상 징후 탐지 및 경보 발령 시 응답 시간 개선	1 분 미만
서비스 고도화	보안 시스템 신뢰성	실시간 보안 모니터링 및 침입 탐지의 위협 탐지율	98% (오탐률 최소화)

1.2. 검증 방안

구분	항목	검증 방안	검증 수행
빌딩 설비 연동/제어	실시간 데이터 업데이트	부하 및 스트레스 테스트	모의 센서 데이터를 이용하여 다수의 기기 연결 시 시스템의 응답 속도 및 안정성 측정
	제어 명령 응답 시간	실제 현장 테스트	파일럿 빌딩에서 실제 설비와 연동 테스트를 수행하여 응답 시간과 데이터 정확성 검증
	시스템 가용성	통합 테스트	IoT 게이트웨이, 제어 API, 대시보드 간의 연계 동작을 시뮬레이션 환경에서 반복 테스트
Asset 자동 동기화	데이터 처리 및 응답 속도	API 성능 테스트	부하 테스트 도구를 활용하여 다수의 동시 요청 시 응답 시간 및 처리량 측정
	데이터 정확성 및 일관성	데이터 검증 테스트	실제 자산 데이터와 시스템에 저장된 데이터를 비교하여 정확도 및 일관성 확인
3D 모델 자동 동기화	3D 모델링 정확도	현장 측정 비교	생성된 3D 모델과 실제 건물의 치수, 구조 요소를 비교 분석하여 정확도 평가
	처리 속도	프로세스 시간 측정	전체 모델링 파이프라인(사진 수집부터 최종 모델 생성까지) 소요 시간 측정
통합 시뮬레이션	에너지 최적화 정확도	에너지 시뮬레이션 검증	과거 에너지 사용 데이터와 비교하여 예측 모델의 오차율 측정 및 검증
	실내환경/연돌 분석데이터 고도화	평가지표 및 센서데이터 비교	ASHRAE Standard 55(열쾌적성 기준) 준수 여부 평가 및 설치된 온·습도 센서데이터 비교
	화재/재난 대피 대응 가이드	화재/재난 시뮬레이션 테스트	가상 재난 상황을 시뮬레이션하여 응답 시간 및 경보 발령 정확도 검증
서비스 고도화	보안 시스템 신뢰성	보안 침투 테스트	모의 침투 테스트(Penetration Test)를 통해 시스템의 취약점 및 탐지율 평가

2. 추진 전략

2.1. 추진 계획

2.1.1 연합 디지털 트윈 기술 개발

□ 기술 개발 정의서 (빌딩 설비 연동 기술)

구 분	내 용
1. 개발 기술	건물 내 주요 설비(HVAC, 조명, 환기 등)를 디지털트윈 플랫폼(BEES)과 연동하여 실시간 모니터링 및 양방향 제어 기술
2. 개발 목적	설비의 운전 상태를 실시간으로 감지하고, 자동제어 시나리오를 적용하여 에너지 효율화 및 쾌적한 실내환경 유지, 비상상황 대응력 향상 도모
3. 개발 기능	- HVAC, 조명, 창호, 환기설비 등의 상태 실시간 조회 - 사용자 재실 여부 및 공간 환경 기반 자동 제어 - 설비 이상 발생 시 경고 및 자동 조치 시나리오 실행 - 설비별 이력 저장 및 유지보수 연계 기능
4. 개발 성과물	
5. 기술 차별성	- 기존의 단방향 설비 모니터링 시스템과 달리, 센서 및 디지털트윈 기반의 양방향 연동 제어와 사용자 상황 인지 기반 자동 제어 시나리오 탑재로 고도화 - 다수 설비 간 통합 관제가 가능한 점에서 기존 BAS 와 차별화
6. 적용 기술	- 표준 설비 통신 프로토콜: BACnet, Modbus, OPC-UA - Edge Device 기반 실시간 데이터 수집 및 명령 송신 - RESTful API 기반 BEES 플랫폼 연동 - 설비 상태 시각화 및 제어 HMI 구축
7. 성능 목표	- 에너지 소비량 절감 운전 시나리오: 기존 대비 10% 이상 절감 기준 운전 - 실시간 데이터 업데이트 : 3 초 이내 - 연동 설비 대응률: 95% 이상 (설계된 연동설비 중 정상작동)

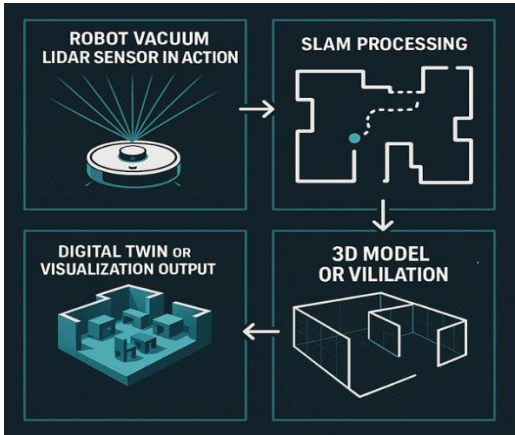
□ 기술 개발 정의서 (빌딩 설비 제어)

구 분	내 용
1. 개발 기술	건물 내 설비(HVAC, 조명, 냉난방 등)에 대해 기계 학습 알고리즘 기반으로 환경·행동 데이터를 분석하여 자동으로 최적 제어를 수행하는 기술
2. 개발 목적	실시간 데이터 기반으로 쾌적성과 에너지 효율을 균형 있게 제어함으로써 운영비용 절감, 설비 수명 연장, 이상 상황 사전 대응을 실현
3. 개발 기능	- 공간별 설비 자동 제어 시나리오 생성 및 실행 - 재실자 수, 활동량, 외기 정보 기반 자동 설정 - 설비 이상 작동 감지 시 예비운전모드 전환 - 에너지 효율/쾌적도 목표에 따른 제어 알고리즘 실시간 보정
4. 개발 성과물	PMV 학습을 통한 자동화 HVAC 제어 하절기 DB 동절기 DB Energy ANN기법 활용 및 상관관계 분석 행위 패턴 유형화 복합데이터 기반 복합설비제어 공간정보 설비정보 조명, HVAC, 냉난방 장치 등 복합제어
5. 기술 차별성	- 단순 온도 기준 제어에서 벗어나 사용자 행동 데이터, 가상센서(CFD 기반 PMV), 환경 변화까지 반영한 지능형 예측 제어 수행 - BAS 수준의 제어를 넘어 다양한 센서/설비 통합제어 가능 - 사용 시나리오 기반 제어 모델 설정 및 자동 전환 기능 지원
6. 적용 기술	- PMV 기반 쾌적도 분석 모델 - 머신러닝 기반 제어 최적화 알고리즘 (의사결정 트리, ANN 등) - 실시간 제어 API 및 MQTT 통신 - Radar 센서, IoT 센서, 가상센서 연동 및 학습 데이터 활용
7. 성능 목표	- 자동 시나리오 적용율: 주요 공간 기준 95% 이상 - 설비 제어 응답 시간: 500ms 이내 - 알림 발생 → 제어 실행 시간: 5 초 이내

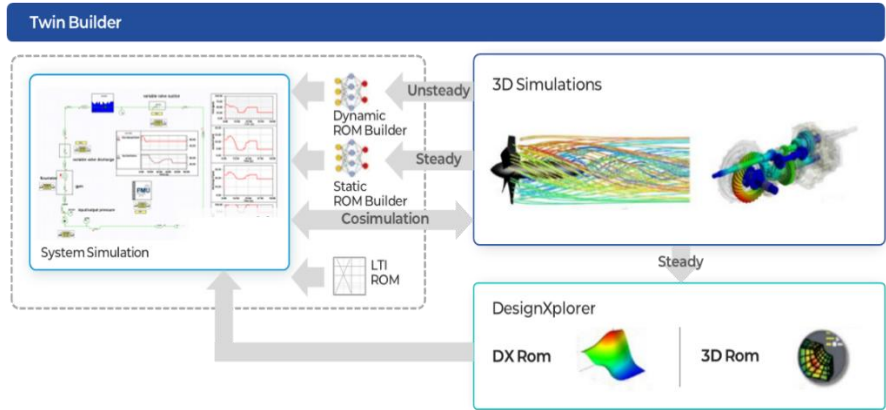
□ 기술 개발 정의서 (Asset 자동 동기화)

구 분	내 용
1. 개발 기술	건물 내 설치된 자재의 고유 정보(종류, 규격, 위치, 수명 등)를 3D 디지털 트윈과 연계하여 실시간으로 조회·분석·예측하며, 로봇청소기·공기청정기 등에 장착된 Radar 센서 기반으로 공간 상태를 실측하고 자동 업데이트하는 기술
2. 개발 목적	수기/정적 정보로 관리되던 자산 정보를 실제 공간 변화와 연동하여 지속적으로 업데이트하고, 자산의 노후화 및 상태 변화를 실시간으로 인지하여 예방적 유지보수 및 수명 주기 기반 관리를 실현
3. 개발 기능	- 자재별 DB 구축 (종류, 규격, 제조사, 설치일, 위치, 이력 등) - 로봇청소기/공기청정기에 장착된 Radar 센서를 통한 실내 스캔 - 자재 위치 및 상태 변화 자동 탐지 → DB 자동 갱신 3D 모델과 연계된 자산 시각화 및 교체 예정 안내 - M/L 기반 노후화 예측 및 유지보수 알림 연계
4. 개발 성과물	 공간 설계 단계: 실제 치수, 수량, 크기 등 구체적인 정보 반영 실제 시공 단계: 실제 시공이 가능한 설계 정보 추가 운영/유지 관리 단계: 시공 이후 유지 관리할 수 있는 정보 반영 노후화 알림 및 유지보수 서비스
5. 기술 차별성	- Radar 센서 기반의 공간 내 자재 상태 변화 인식 및 업데이트 자동화 - 사진 기반 3D 모델 및 자산 DB 자동 연계로 시각적 추적 및 관리 - M/L 기반 자산 상태 분석 및 수명 예측으로 정비 시기 선제적 판단 가능
6. 적용 기술	- Radar 센서 기반 PCD 분할 및 상태 변화 인식 알고리즘 - 사진 기반 SfM/MVS 3D 모델링 + Semantic 속성 연계 - M/L 기반 회귀분석/시계열 분석 알고리즘으로 수명 예측 - 모바일 디바이스 기반 지속형 데이터 수집 파이프라인 - Web 기반 자산관리 대시보드 및 RESTful API
7. 성능 목표	- 자산 위치 자동 매핑 처리속도: 300ms 미만 3D 연계 시각화 정확률: 95% 이상 - 자산정보 자동 업데이트 반영률: 95% 이상

□ 기술 개발 정의서 (3D 모델 자동 동기화)

구 분	내 용
1. 개발 기술	실내외 공간의 다각도 이미지를 수집하여 SfM(Structure from Motion), MVS(Multi-View Stereo) 기술로 3D 모델을 자동 생성하고, 이를 3D 디지털 트윈 모델에 연계하는 기술
2. 개발 목적	BIM 모델 등과 연계하여 실내 공간의 실제 상태를 시각화하고, 공조 해석, 화재 시뮬레이션, 자산관리 등 다양한 기능과 통합가능한 동적 디지털 트윈 환경 구성
3. 개발 기능	- 사진 기반 자동 3D 모델 생성 (SfM/MVS 알고리즘 활용) - 의미 기반 PCD(Point Cloud Data) 분할 및 객체 태깅 - 실내 자산 정보(BIM 또는 자산 DB)와의 정합화 - 이동형 로봇(청소기, 공기청정기 등) 기반 공간 학습 데이터 수집 - 모델 자동 업데이트 및 실시간 정합률 점검
4. 개발 성과물	
5. 기술 차별성	- 기존 2D 도면이나 BIM 이 제공하지 못하는 실내 상세 환경의 시각적 재현 - 로봇청소기·공기청정기 등의 모바일 디바이스 + Radar 센서를 활용한 지속적 데이터 수집 및 자동 갱신 3D 모델에 의미 정보(Semantic)를 추가하여 자산관리·시뮬레이션 연계까지 확장 가능
6. 적용 기술	- SfM, MVS 기반 사진 정합 및 깊이 추정 알고리즘 - Radar, Lidar 센서 기반 거리 보정 및 영상-공간 융합 - RGB-D 기반 상태 인식 및 분할(딥러닝) - M/L 기반 지속 학습 파이프라인 (이동형 촬영 데이터 활용) 3D 포맷 변환 및 WebGL 기반 시각화(gITF, OBJ 등)
7. 성능 목표	3D 모델 정합률: $\pm 5\%$ 이내 (BIM 대비) - 자동 모델 생성 시간: 30 분 이내(건물규모별 데이터처리속도에 따라 증가가능)

□ 기술 개발 정의서 (통합 시뮬레이션)

구 분	내 용
1. 개발 기술	CFD, 에너지, 화재, 피난 등 이기종 시뮬레이터 간의 데이터를 연동·변환하여 하나의 통합된 시뮬레이션 환경을 구성하고, 물리적 상호작용을 반영한 복합 시나리오 해석을 수행하는 기술
2. 개발 목적	개별 시뮬레이터의 한계를 극복하고, 현실성 높은 결과값과 복합 이벤트 예측을 기반으로 건축 설계, 시설 운영, 재난 대응 등 의사결정 지원을 실현
3. 개발 기능	- CFD→에너지, 피난→화재 시나리오 흐름 기반 데이터 연동 - 시뮬레이션 조건 통합 설정 인터페이스 - 물리적 상호작용 고려한 다중 시나리오 구성 기능 - 시뮬레이션 결과 3D 시각화 및 경과 비교 분석 - 통합 대시보드 및 결과 보고서 자동 생성
4. 개발 성과물	 The diagram illustrates the 'Twin Builder' system architecture. It shows a 'System Simulation' block on the left, which interacts with 'Dynamic ROM Builder' and 'Static ROM Builder' blocks in the center. These builders feed into 'Cosimulation' and 'LTI ROM' blocks. The 'Cosimulation' block then feeds into '3D Simulations' on the right. The '3D Simulations' block outputs to 'DesignXplorer', which generates 'DX Rom' and '3D Rom' results. Arrows indicate the flow of data and simulation results between these components, with labels for 'Unsteady', 'Steady', and 'Steady' states.
5. 기술 차별성	- 기존에는 각 시뮬레이터가 독립적으로 작동했지만, 본 기능은 미들웨어 기반 자동 연동을 통해 시뮬레이터 간 데이터 흐름을 통합 - CFD 기반 공조해석 결과를 에너지 시뮬레이션 통합 - 화재 확산 예측 시뮬레이션 기반 피난 경로 분석 통합
6. 적용 기술	- OpenFOAM, CONTAM, EnergyPlus, FDS-EVAC 등 시뮬레이터 연동 API - XML/JSON 기반 시뮬레이션 조건 변환 인터페이스 - 데이터 정규화 및 좌표계 일치 처리 알고리즘 - Web 기반 통합 대시보드 및 시각화 뷰어 - 시뮬레이션 자동 배치 실행 및 결과 추적 시스템
7. 성능 목표	- 시뮬레이터 연동 시나리오 연계: 최소 2 개 이상 - 에너지 예측 오차율: 5% 미만

2.1.2 BEES 플랫폼 서비스 고도화

□ 서비스 개발 정의서 (지능형 공간 관리)

구 분	내 용
1. 개발 서비스	실내에 실제 사람이 있는지(재실) 여부와 환경 상태를 실시간으로 파악하여, 에너지 절감 모드·재실/비재실 모드·계절 모드·쾌적 모드 등 다양한 운영 모드를 자동 전환함으로써 건물의 에너지 효율, 공간 쾌적성, 활용성, 환경 영향을 종합 최적화하는 기능
2. 개발 목적	건물의 공간별 에너지 절감, 쾌적성 유지, 공간 활용성 극대화, 환경 영향 저감을 위한 지능형 통합 공간 관리
3. 개발 기능	- 재실/비재실 모드 : 모션·CO₂ 센서로 재실 유무 감지 → 조명·HVAC 자동 제어 - 에너지 절감 모드 : 비사용 시간대 온도 셋포인트 완화, 조명 밝기 최소화 - 계절 모드 : 외기 온·습도·태양 복사량 분석 → 냉·난방 알고리즘 최적화 - 쾌적 모드 : PM2.5·VOC 센서 연동 환기 제어, 온·습도·조도 자동 유지 - 프리쿨/프리히트 : 심야 전력 요금·외기 조건 활용 선제 냉난방 - 스마트 스케줄링 : 일일/주간 예약 정보 기반 모드 자동 스케줄링 - 대시보드 & 알림 : 운영 현황 시각화, 이상 상황 푸시 알림
4. 개발 성과물	
5. 기술 차별성	- 에너지 절감 모드, 재실/비재실 모드, 계절 모드, 쾌적 모드 등 복합 모드 동시 운용을 위한 다중 모드 통합 운영 - 재실 패턴, 공간 환경 정보, 외부 기상 예보 기반 선제적 모드 전환 기능 적용을 위한 M/L 엔진 적용 - 개별 실, 구역별 선호 온·습도 프로파일 관리를 통한 사용자 맞춤화 기능 제공 - 센서 데이터에 대한 실시간 피드백 루프 시스템을 통해 지속적인 학습 및 고도화로 자동 성능 최적화 구현
6. 적용 기술	- 센서 네트워크: 모션, CO₂, 온·습도, 조도, 공기질 - IoT 통합 플랫폼: MQTT/REST API 기반 데이터 송수신 - AI/ML 엔진: 재실 예측, 에너지 사용 최적화 모델 - 빌딩관리시스템(BMS) 연동: BACnet, Modbus, KNX 프로토콜 - 클라우드/엣지 컴퓨팅: 실시간 제어·로컬 예측 알고리즘 - 데이터 분석 & 시각화: Grafana/Superset 기반 대시보드

7. 성능 목표	- 에너지 소비 절감율 : 10% 이상 (연간 기준) - 실내 쾌적도 유지 시간 비율 : 80% (설정 온도 범위 내) - 공간 활용 효율 향상율 : 20% 이상 (실제 사용률 개선) - 모드 전환 지연 시간 : 5 분 이하
----------	--

□ 서비스 개발 정의서 (Multi-Tenant)

구 분	내 용
1. 개발 서비스	디지털트윈 플랫폼 상에서 설비 상태, 에너지 소비, 공간 환경 변화 등을 실시간으로 모니터링하고, 건물 운영/관리 이해 관계자(건물주, 운영자, 입주자)에게 관련 정보 제공 및 이상 징후 발생 시 자동으로 경고 알림을 전송하는 서비스
2. 개발 목적	건물 내 설비/환경 변화에 신속히 대응하여 운영 안정성 향상, 유지보수 비용 절감, 에너지 낭비 방지 등 능동적 건물 관리 체계 실현
3. 개발 기능	- 설비 상태 실시간 모니터링 (온도, 습도, 진동, 소비전력 등) - 이상 상태(PMV 이탈, 과부하, 노후화 등) 감지 시 경고 - 사용자별 알림 설정(설비 유형, 공간, 이벤트 종류 등) - Co-pilot 기반 대응 가이드 자동 생성 (예: 조치 매뉴얼, 담당자 알림)
4. 개발 성과물	 The screenshot displays a comprehensive digital twin platform interface. It features a central 3D city model with highlighted buildings. Surrounding this are multiple data dashboards: a top-left bar chart for energy consumption, a top-right map of Korea, a middle-left donut chart for PMV status, and a bottom-left line graph for environmental trends. On the right side, there are several smaller panels showing building details, energy usage, and a heatmap. The interface is modern and data-driven, designed for real-time monitoring and management.
5. 기술 차별성	- 단순 텍스트 기반 경고에서 벗어나 3D 디지털 모델 기반 시각적 경고 위치 표시 - IoT 센서 + 가상 센서(CFD/M/L 예측 등) 연계로 이상 상황 사전 탐지 - 사용자 맞춤형 알림 기준 설정 및 Co-pilot 형 스마트 안내 기능 제공
6. 적용 기술	- 실시간 센서 스트림 처리 기술 (MQTT, WebSocket) - 이상 탐지 M/L 모델 (변칙 탐지, 임계치 예측 등) 3D 기반 시각화 엔진 (Three.js, glTF 등) - 사용자 맞춤형 Rule 기반 이벤트 트리거 시스템 - 모바일 Push / Web Notification 다채널 알림
7. 성능 목표	- 사용자별 알림 설정 반영률: 95% 이상 - Co-pilot 가이드 제공 성공률: 95% 이상

2.2 추진 방법

2.2.1 업무 수행 내용

구분	수행 내용
1. 요구사항 정의 및 분석	- 연합 디지털 트윈 기술 개발 명세서 작성 - BEES 플랫폼 서비스 고도화 명세서 작성 - 공통 데이터 포맷, API 규격, 보안 및 인증 체계, 데이터 교환 등 서비스 간 인터페이스 및 의존성 분석
2. 시스템 아키텍처, 알고리즘 설계	- 마이크로서비스(MSA) 기반 플랫폼 아키텍처 설계 (공통 인프라, API 게이트웨이, 메시지 브로커, 보안, 데이터베이스 등) - BEES 플랫폼 서비스 및 UI&UX 설계 - API, 데이터 형식, 인증/인가, 메시지 전송 규약, 보호 정책 확정
3. 개발 기술 단위 테스트	- 개발 기술 유닛 테스트 및 코드 리뷰, CI 도구(Jenkins, GitLab CI) 통한 자동화 테스트 - 스프린트 리뷰 및 피드백 - 2~4 주 단위 스프린트로 진행, 각 스프린트마다 산출물 점검 및 개선사항 반영
4. 개발 기술 연동 테스트	- 전체 시스템의 통합 빌드 및 자동 배포 파이프라인 설계 - 서비스 간 호출, 데이터 흐름, 보안 인증, 테스트 - 모듈 간 충돌, 데이터 불일치, 인터페이스 오류 문제점 도출
5. 종합 시스템 테스트 및 품질 검증	- 종합 테스트 계획 수립: 기능, 시스템 통합, 부하, 성능, 보안, UAT 테스트 케이스 작성 - 자동화 및 수동 테스트 도구 활용하여 전 서비스 기능 검증 - 테스트 결과에 따른 결함 사항 도출 및 회귀 테스트 진행 - 최종 품질 검증: 요구사항 및 성능/보안 기준 충족 확인
6. 시범 운영	- 시범 운영(Pilot) 실시: 이해 관계자 그룹 대상으로 시범 운영 실시 - 운영 환경 구축, 데이터 마이그레이션, 백업/복구 체계 점검 - 운영 매뉴얼, 사용자 교육 자료, 장애 대응 프로세스 마련 - 최종 회귀 테스트, 운영 인수인계, 배포 리허설 진행 - 전 서비스 최종 안정성 확인 후 일괄 배포(go-live) 준비

2.3.2. 개발 내용

□ 설비 연동/제어 시스템 개발

구분	내용	
개발 요소	구분	설명
	설비 통신 게이트웨어	BACnet/Modbus, UA/MQTT 통신을 수신/변환하여 내부 메시지 버스로 전달
	설비 연동 어댑터	설비별 맞춤 프로토콜 처리 및 데이터 표준화 처리 모듈
	설비 제어 API	제어 명령을 검증·로그 처리 후, 게이트웨이를 통해 설비로 전송
	설비 제어 로직 엔진	스케줄, 재실/비재실 모드, 에너지 절감 등 자동 제어 시나리오 정의 및 실행
	디지털 트윈 연계 모듈	설비 상태 변경을 디지털 트윈 3D 모델에 실시간 반영
기술 스택	- 게이트웨이/어댑터: [NubeIO, OpenHAB, EdgeX Foundry], 또는 Node.js 기반 커스텀 개발 - 프로토콜 지원: BACnet4J (Java), libmodbus (C) - MQTT Broker: Eclipse Mosquitto, EMQX - 제어 API 서버: Spring Boot, Node.js (NestJS) - 상태 시각화: Grafana + WebSocket 기반 Custom Dashboard	
업무 내용	개발 단계	개발 업무
	1 단계	설비별 통신 프로토콜(BACnet/Modbus) 분석 및 연동 범위 정의
	2 단계	통신 게이트웨이 및 설비 어댑터 설계 및 구현
	3 단계	설비 데이터 실시간 수집 로직 구현 (상태 모니터링 포함)
	4 단계	제어 API 및 원격 제어 명령 처리 모듈 구현
	5 단계	제어 시나리오 정의 엔진 구현 (재실/비재실, 에너지 절감 모드 등)
	6 단계	디지털 트윈 3D 모델과의 연계 및 상태 반영 시각화 구현
	7 단계	통신 실패/오류 대비 예외 처리 및 제어 안전성 검증
알고리즘	<pre> graph LR subgraph Equipment [설비
(센서 & 장비)] direction TB BACnet Modbus KNX end subgraph Gateways [설비통신 게이트웨어] direction TB BACnet Modbus KNX end subgraph Adapters [설비연동
어댑터] direction TB Adapter end subgraph DataFlow [데이터 수집 모듈
전처리 및 필터링] direction TB DataFlow end subgraph DB [시계열 DB] direction TB DB end subgraph API [제어 API] direction TB API end subgraph Logic [제어 로직 엔진] direction TB Logic end subgraph Hub [통합 데이터 허브] direction TB Hub end subgraph DT [디지털 트윈 3D 모델] direction TB DT end Equipment <--> Gateways Gateways <--> Adapters Adapters <--> DataFlow DataFlow <--> DB DB <--> Hub Hub <--> Logic Logic <--> API API <--> Equipment Logic <--> DT DT <--> API </pre>	

□ Asset 자동 동기화 시스템 개발

구분	내용	
개발 요소	구분	설명
	자산 DB	자재 정보 (종류, 규격, 수명, 설치 위치, 교체 이력 등) 저장
	3D 모델 연계 모듈	자재 정보와 BIM/3D 모델의 위치 정보 매핑
	수명 예측 엔진	통계 + 머신러닝 기반으로 자산 상태 분석 및 수명 예측
	예방 보수 추천 서비스	교체 시기 예측 및 관리자 알림 자동화
기술 스택	- DB: PostgreSQL + PostGIS (공간정보 저장용) - 백엔드: Django REST Framework 또는 Spring Boot - 수명 예측 엔진: Python (scikit-learn, Prophet 등) - 시각화 대시보드: Apache Superset, React + Chart.js - 3D 연계: BIM 요소 ID 매핑 → Three.js / glTF 기반 뷰어와 연동	
업무 내용	개발단계	개발업무
	1 단계	자산 항목 정의 및 메타데이터 설계 (자재 종류, 수명, 설치 위치 등)
	2 단계	자산 DB 스키마 설계 및 CRUD 기능 개발
	3 단계	자산 정보와 3D 모델 요소 간 매핑 인터페이스 개발
	4 단계	자산 노후화 및 수명 예측 로직 개발
	5 단계	관리자용 자산 현황/보수계획 대시보드 구축
알고리즘	<pre> graph TD subgraph TopBox [] direction LR A[설계/BIM 데이터] --> B[초기 자산 등록] B --> C[자산관리 DB] C --> D[정비 이력 업데이트] end subgraph BottomBox [] direction LR E[수명예측엔진] --> F[유지보수 추천/알림] end G[통합 데이터 허브] <--> 내부 API 호출 H[건물 유지보수시스템] H --> A H --> B H --> C H --> E I[디지털 트윈 3D 모델] <--> H </pre>	

□ 3D 모델 자동 동기화 시스템 개발

구분	내용	
개발 요소	구분	설명
	사진 업로더/관리기	사진 메타데이터 수집 및 전처리
	SfM (Structure-from-Motion)	이미지로부터 카메라 위치 추정 및 포인트 클라우드 생성
	MVS (Multi-View Stereo)	고밀도 점군 재구성
	메쉬 생성기 / 텍스처 매퍼	점군 → 표면 메쉬 → 실사 텍스처 맵핑
	의미론적 분할기 (딥러닝)	벽/문/창문/바닥 등의 요소 자동 식별
	BIM 통합 모듈	기존 BIM 모델과 정합성 확보 및 포맷 변환 (OBJ/FBX/gITF)
기술 스택	- Photogrammetry 툴킷: OpenMVG + OpenMVS or COLMAP - 의미론적 분할: Detectron2, SegFormer (PyTorch 기반) - 후처리 및 뷰어: MeshLab, Potree (Web-based point cloud viewer) - 포맷 지원: glTF, OBJ, FBX - 좌표 정합: QGIS, PDAL	
업무 내용	개발단계	개발업무
	1 단계	입력 이미지 전처리 파이프라인 구성
	2 단계	SfM + MVS 기반 3D 포인트 클라우드 생성 알고리즘 적용
	3 단계	메쉬 생성 및 텍스처 매핑 자동화
	4 단계	딥러닝 기반 구조요소 인식(semantic segmentation) 적용
	5 단계	모델 스케일 정합/좌표 정합(지오레퍼런싱) 구현
	6 단계	기존 BIM 데이터와 통합 및 표준 포맷 export 기능 개발
알고리즘	<pre> graph TD BIM[건물자산관리시스템] -- "정비 이력 업데이트" --> Hub[통합 데이터 허브] Hub -- "BIM 통합" --> Photos[촬영 사진] Photos --> SfM[SfM] SfM --> MVS[MVS] MVS --> Model[3D 모델] Model --> Seg[딥러닝 기반 요소 분할] Seg --> Mesh[메쉬 생성 + 텍스처] Mesh --> DT[디지털 트윈 3D 모델] DT --> Hub Hub -- "고장/경보 이벤트 전송" --> Maint[유지보수 이벤트 수신 모듈] Maint --> Sched[작업 타겟 생성 및 스케줄링] Sched -- "현장 정비" --> Work[정비 작업 모듈] Work -- "완료 보고서 전송" --> Hub </pre>	

□ 시뮬레이터 연동 및 통합

구분	내용	
개발 요소	구분	설명
	시뮬레이션 통합인터페이스	각 시뮬레이터와의 데이터 송수신 표준화 API
	시뮬레이션 데이터 브로커	입력/출력 데이터를 변환하여 시뮬레이터 간 전달
	시뮬레이션 관리 서비스	시뮬레이션 요청, 스케줄링, 상태 모니터링
	시뮬레이션 결과 시각화 모듈	연동효과, 화재, 피난 결과를 3D 디지털 트윈에 반영
기술 스택	- 통합 API 서버: Python (FastAPI) 또는 Java (Spring Boot) - 시뮬레이터 연동: 외부 실행 via CLI / REST API / Python Wrapper (EnergyPlus, FireDynamicsSimulator 등) - 분산 큐: Celery + Redis or RabbitMQ - 데이터 브로커: Kafka (시뮬레이션 간 이벤트 전달) - 시각화: CesiumJS, Three.js, or Unity 기반 Web Viewer	
업무 내용	개발단계	개발업무
	1 단계	연동 대상 시뮬레이터(연동효과/화재/피난)별 입출력 데이터 구조 분석
	2 단계	시뮬레이션 통합 인터페이스 정의 및 API 개발
	3 단계	시뮬레이션 요청 관리, 상태 모니터링 및 결과 수집 로직 구현
	4 단계	시뮬레이션 간 결과 연계 분석 엔진 개발
	5 단계	결과 데이터를 디지털 트윈 모델에 시각화 통합
	6 단계	병렬 시뮬레이션 실행/큐 관리/로깅 기능 구현
알고리즘	운영자 UI(웹) 현장 시설/장비 건물통합관제시스템 운영관리 어플리케이션 내부 API 호출 스케줄 업데이트 통합 데이터 허브 디지털 트윈 3D 모델 디지털 트윈 환경 데이터 화재/피난 시뮬레이터 결과 정보 CFD 시뮬레이션 분석 피난 시뮬레이터 화재 시뮬레이터 시나리오 정의 모듈 제어 명령 전달 제어 명령 입력 상태 업데이트 조회 피드백 데이터 전송	

□ 지능형 공간 관리

구분	내용	
개발 요소	구분	설명
	센서 네트워크	- 모션·CO ₂ ·온·습도·조도 센서 - PoE 스위치 및 Wi-Fi AP
	로컬 서버	- MQTT 브로커 및 수집 에이전트 * 시계열 DB, 관계형 DB, AI 인퍼런스 서비스 - BMS 제어 모듈
	AI 예측·제어 엔진	- 재실 예측 모델 (LSTM, RandomForest) - 에너지 수요 최적화 알고리즘 (선형계획법, 강화학습)
	BMS 연동 모듈	- BACnet/Modbus/KNX 통신 어댑터 - HVAC·조명 제어 API 서버
	운영 대시보드	- Grafana 실시간 모드·환경 지표 시각화 - 관리자 UI (모드 커스터마이징, 스케줄 관리)
기술 스택	- 센서 : 모션·CO₂·온·습도·조도 센서 - 네트워크 : Ethernet (PoE 스위치) / Wi-Fi, VLAN, 방화벽 규칙 설정 - 로컬 서버 : 서버 HW - Intel Xeon 8 코어, 32 GB RAM, SSD 1TB 이상급, OS - Ubuntu 24.04 LTS - 미들웨어 : MQTT 브로커 - Eclipse Mosquitto, 데이터 수집 에이전트 (Python) - 데이터베이스 : 시계열 DB - TimescaleDB / InfluxDB, 관계형 DB - PostgreSQL - 백엔드 : API-Python FastAPI / Node.js Express, AI/ML: TensorFlow, scikit-learn - 제어 연동 : BMS 프로토콜: BACnet, Modbus, KNX, 제어 모듈 (gRPC/REST) - 프론트엔드 : React / Vue.js, 대시보드 - Grafana / Superset - CI/CD : Ansible / Terraform (로컬 네트워크 인프라 구성), GitLab CI	
업무 내용	개발단계	개발업무
	1 단계	- 기능·비기능 요구사항 수집·정리 - KPI 및 성과지표(에너지 절감율, 쾌적도 유지 등) 확정
	2 단계	- 모드(재실/비재실, 계절, 쾌적, 절전 등)별 동작 로직 설계 - 데이터 모델링(센서·사용자·스케줄 테이블) - 통합 아키텍처·네트워크 설계(LAN, 방화벽)
	3 단계	- 센서 네트워크 연동 PoC (모션·CO ₂ ·온·습도·조도) via 로컬 게이트웨이 - 로컬 서버 MQTT 브로커 설치·검증 - 간이 대시보드 구현

업무 내용	<table border="1"> <thead> <tr> <th>개발단계</th><th>개발업무</th></tr> </thead> <tbody> <tr> <td>4 단계</td><td> - IoT 데이터 수집 모듈 개발 (센서 → 로컬 서버) - AI 예측 엔진 개발·학습 파이프라인 로컬 배포 - BMS 제어 API 연동 모듈 개발 </td></tr> <tr> <td>5 단계</td><td> - End-to-End 테스트 (모드 전환 시나리오) - 부하·장애·보안(네트워크·서버) 테스트 - 성능 튜닝(서버 처리량, 레이턴시) </td></tr> <tr> <td>6 단계</td><td> - 로컬 인프라(서버·네트워크) 배포 자동화 스크립트 작성 - 모니터링, 버그 수정 </td></tr> <tr> <td>7 단계</td><td> - 운영 지표 모니터링·리포팅 - AI 모델 재학습·정밀도 개선 - 추가 모드(공기질 위험 모드) 확대 적용 </td></tr> </tbody> </table>	개발단계	개발업무	4 단계	- IoT 데이터 수집 모듈 개발 (센서 → 로컬 서버) - AI 예측 엔진 개발·학습 파이프라인 로컬 배포 - BMS 제어 API 연동 모듈 개발	5 단계	- End-to-End 테스트 (모드 전환 시나리오) - 부하·장애·보안(네트워크·서버) 테스트 - 성능 튜닝(서버 처리량, 레이턴시)	6 단계	- 로컬 인프라(서버·네트워크) 배포 자동화 스크립트 작성 - 모니터링, 버그 수정	7 단계	- 운영 지표 모니터링·리포팅 - AI 모델 재학습·정밀도 개선 - 추가 모드(공기질 위험 모드) 확대 적용
개발단계	개발업무										
4 단계	- IoT 데이터 수집 모듈 개발 (센서 → 로컬 서버) - AI 예측 엔진 개발·학습 파이프라인 로컬 배포 - BMS 제어 API 연동 모듈 개발										
5 단계	- End-to-End 테스트 (모드 전환 시나리오) - 부하·장애·보안(네트워크·서버) 테스트 - 성능 튜닝(서버 처리량, 레이턴시)										
6 단계	- 로컬 인프라(서버·네트워크) 배포 자동화 스크립트 작성 - 모니터링, 버그 수정										
7 단계	- 운영 지표 모니터링·리포팅 - AI 모델 재학습·정밀도 개선 - 추가 모드(공기질 위험 모드) 확대 적용										
알고리즘	<pre> graph TD A[센서 디바이스] -- "Ethernet/WiFi" --> B[IoT 게이트웨이 스위치] B <--> C[관리·운영 워크스테이션] B --> D[MQTT 브로커] D -- "MQTT(over TLS)" --> E[데이터 수집·처리 서버] E -- "내부 API 호출" --> F[시계열 DB TimescaleDB] E -- "내부 API 호출" --> G[데이터 백업 스크립트] E -- "내부 API 호출" --> H[관계형 DB PostgreSQL] E -- "내부 API 호출" --> I[사용자 스케줄 테이블] F -- "JDBC/ODBC" --> J[AI 예측 제어 서버] H -- "SQL" --> J J -- "제어 명령" --> K[BMS 제어] K --> L[HVAC(냉난방, 공조), 조명, 환기 설비] K --> M[디지털 트윈 대시보드 인증] </pre> The diagram illustrates the system architecture. It starts with '센서 디바이스' (Sensor Devices) connected via 'Ethernet/WiFi' to an 'IoT 게이트웨이 스위치' (IoT Gateway Switch). This switch is bidirectionally connected to a '관리·운영 워크스테이션' (Management/Operation Workstation). Data flows from the gateway switch to an 'MQTT 브로커' (MQTT Broker) and then via 'MQTT(over TLS)' to a '데이터 수집·처리 서버' (Data Collection/Processing Server). This server has internal API calls to a '시계열 DB (TimescaleDB)', '데이터 백업 스크립트' (Data Backup Script), '관계형 DB (PostgreSQL)', and a '사용자 스케줄 테이블' (User Schedule Table). The '시계열 DB' and '관계형 DB' connect to the 'AI 예측 제어 서버' (AI Prediction/Control Server) using 'JDBC/ODBC' and 'SQL' respectively. The AI server, which includes 'TensorFlow Serving / FastAPI', '재실 예측 모델, 수요 최적화 모델' (Real-time prediction model, demand optimization model), and '배치 학습 스케줄러' (Batch learning scheduler), sends '제어 명령' (Control commands) to the 'BMS 제어' (BMS Control) block. The BMS control block contains 'BACnet/Modbus/KNX 어댑터', 'REST API', and '제어 로직 (Setpoint, on/off)'. Finally, the BMS control block is connected to 'HVAC(냉난방, 공조), 조명, 환기 설비' (HVAC, lighting, ventilation equipment) and a '디지털 트윈 대시보드 (인증)' (Digital Twin Dashboard (Authentication)).										

□ Multi-Tenant 서비스

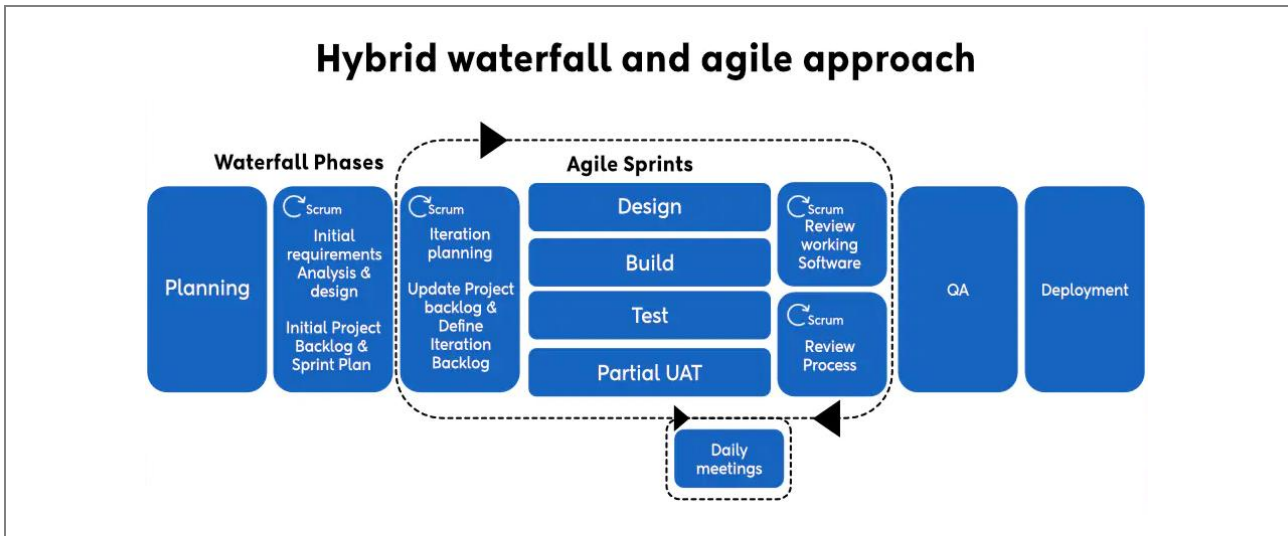
구분	내용	
개발 요소	구분	설명
	시계열 데이터베이스 (e.g., InfluxDB, TimescaleDB)	- 센서 및 설비 운영 이력 저장
	데이터 전처리/정제 모듈	- 노이즈 필터링, 이상값 처리
	데이터 분석/예측 엔진	- 설비 이상 탐지, 고장 예측, 에너지 분석
	API 게이트웨이	- 외부 연동 시스템과 통신 관리
	데이터 표준 포맷 정의	- JSON/CSV 기반 통일된 데이터 모델 제공
기술 스택	- 시계열 DB: InfluxDB, TimescaleDB (PostgreSQL 기반) - ETL 파이프라인: Apache NiFi, Logstash, custom Python - 분석 도구: Jupyter, pandas, Prophet, PyCaret - 시각화: Grafana, Redash - 로그 저장소: Elasticsearch + Kibana (ELK 스택)	
업무 내용	개발단계	개발업무
	1 단계	센서/설비/시뮬레이션 데이터의 수집 포맷 정의 및 표준화
	2 단계	시계열 DB 설계 및 저장 로직 구현
	3 단계	데이터 필터링, 클렌징, 이상값 탐지 기능 구현
	4 단계	데이터 기반 예측 모델(에너지 소비, 고장 예측 등) 개발
	5 단계	시각화 및 리포팅 자동화 기능 구현
알고리즘	<pre> graph TD UI[사용자 UI] <--> DT3D[디지털 트윈 3D 모델] UI --> AG[API Gateway] AG --> S[설비제어 서비스] AG --> DT3D S --- SS[시뮬레이터 서비스] SS --- AS[자산관리 서비스] AS --- 3DS[3D 모델링 서비스] S --- BLS[보안/로그 서비스] SS --- BLS AS --- BLS 3DS --- BLS BLS --> TDH[통합 데이터 허브] TDH --- TSDB[시계열 DB] TDH --- FS[파일 스토리지] TDH --- MB[메세지 브로커] </pre>	

□ 보안 및 인증체계 구축

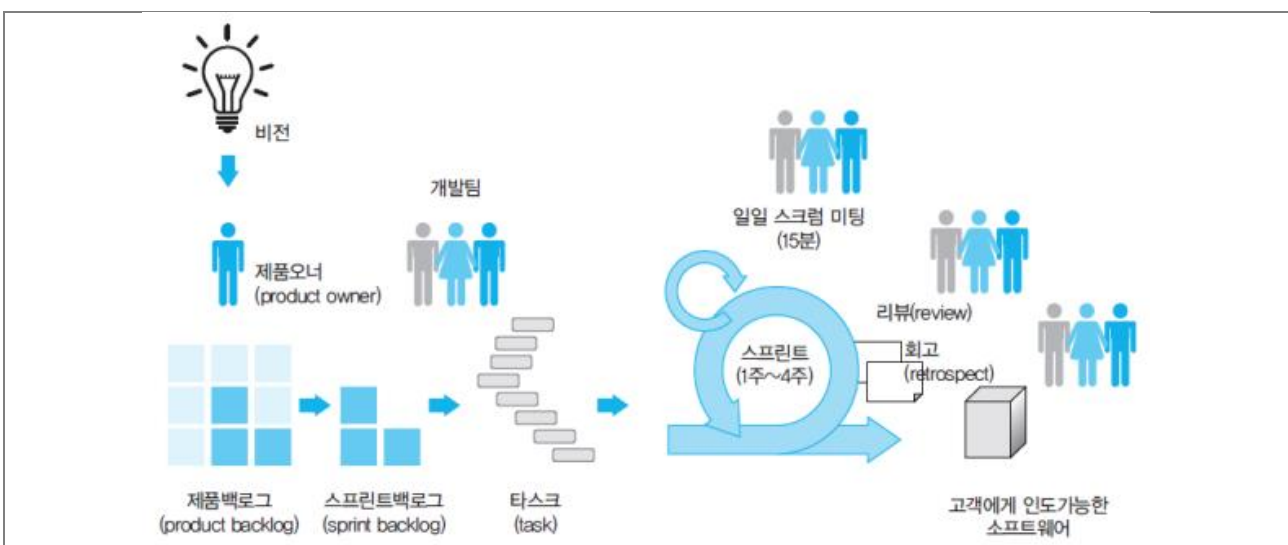
구분	내용	
개발 요소	구분	설명
	API 보안 게이트웨이	인증 토큰 및 요청 유효성 검증
	역할 기반 접근 제어 (RBAC)	관리자, 운영자, 외부 연동 별 권한 구분
	엔드-투-엔드 암호화	TLS/HTTPS + 제어 명령 디지털 서명
	침입 탐지 및 이상 제어 차단 시스템	시그니처 기반 탐지 + AI 기반 행위 분석
	감사 로그 시스템	제어 이력 변조 불가 저장 및 검색 기능
기술 스택	- 인증/인가: Keycloak, Spring Security, OAuth2 Server (JWT 기반) - 암호화: TLS (OpenSSL 기반), AES256 + SHA256 - 이상탐지: Wazuh, Suricata + ELK 연계 - 감사 로그: PostgreSQL + Immutable logging with Blockchain-style hashes (Hashchain 방식)	
업무 내용	개발단계	개발업무
	1 단계	사용자 역할 정의 및 역할 기반 접근 제어(RBAC) 시스템 설계
	2 단계	API 인증/인가 구현 (JWT + OAuth2 기반)
	3 단계	제어 명령 검증 및 암호화 처리 로직 개발
	4 단계	비정상 명령 탐지 및 이상행위 알림 시스템 개발
	5 단계	감사 로그 저장 시스템 및 관리자 UI 구축
알고리즘	<pre> graph TD User[사용자/클라이언트 (웹/IoT)] --> Gateway[API Gateway (WAF, 토큰 검증)] Gateway --> Auth[인증/인가 서버 OAuth2, SSO, JWT 발급] Gateway --> Service1[내부 서비스 1 Microservices 토큰 검증] Gateway --> Services2[내부 서비스 2, 3, 4 RBAC/ABAC] Auth --> SIEM[로그/감사 시스템 데이터 스토리지 SIEM, Audit Log] Service1 --> SIEM Services2 --> SIEM </pre>	

2.3.3 개발 방법론

□ 개발 방식: 하이브리드 개발 방법론 (워터폴+애자일)



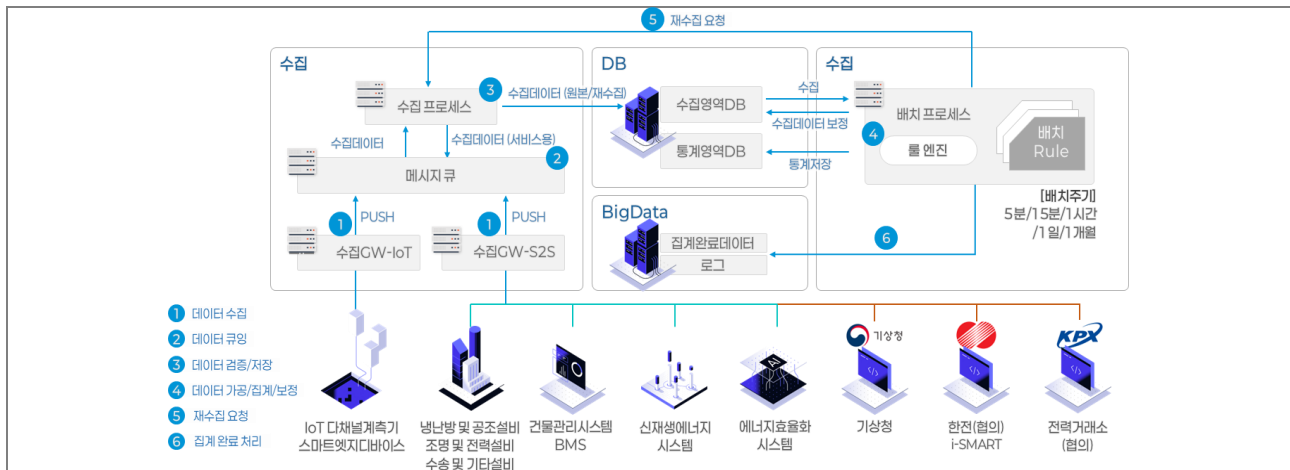
- 본 연구개발 과제는 14 개월 내 연합 디지털 기술 개발 및 서비스 고도화를 수행하여 한 번에 통합 배포해야 하는 집중개발 과제로서, 초기 계획과 일정 관리의 예측성이 중요한 워터폴 방법과 개발 과정의 유연성과 신속한 피드백이 강점인 애자일(Scrum) 방법의 결합이 적합
- 구체적으로, 요구사항 정의 및 설계 단계에서는 워터폴 방식으로 명확한 계획과 산출물을 확정하고, 개발 및 테스트 단계에서는 스프린트 단위의 애자일(Scrum) 방법을 활용해 반복적 개발과 조기 테스트를 수행
- 이러한 “Plan with Waterfall, execute with Agile” 접근법을 통해 초기 단계에서는 전체 범위, 일정, 예산을 안정적으로 계획하고, 이후 단계에서는 변화 대응과 품질 개선을 유연하게 진행 가능



- 이를 실행하기 위해 스크럼 프레임워크를 도입하며, 2 주~4 주의 짧은 스프린트를 반복하면서 각 서비스별로 기능을 구현하고, 스프린트 종료 시마다 제품 증분(increment)을 시연 및 검토

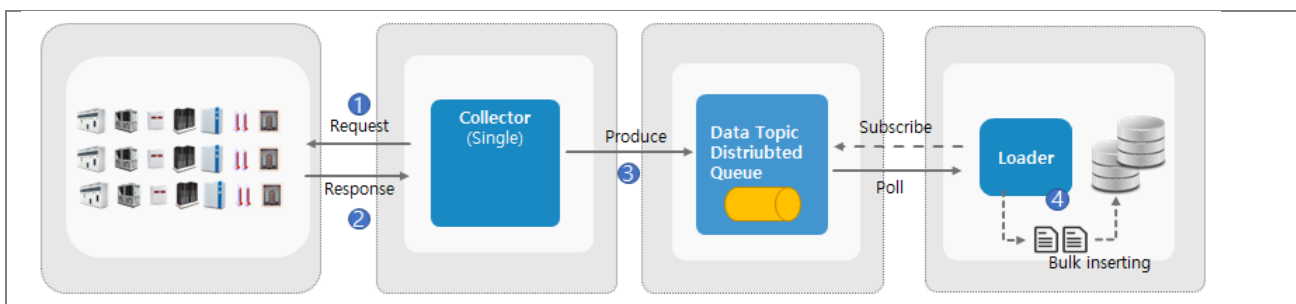
- 이렇게 애자일 기법을 활용하되, 상위 일정 관리 측면에서는 주요 마일스톤과 단계별 산출물을 설정하여 통제
- 즉, 요구사항 명세서 승인, 아키텍처 설계 완성, 통합 테스트 완료 등 주요 지점에서는 워터폴식 단계 게이트를 두어 품질을 확인하고 다음 단계로 넘어가며, 이와 같은 하이브리드 방법론 적용으로 일정 준수와 품질 확보 모두 달성

□ 전체 처리 체계 프레임: 데이터 신뢰성 기반 디지털트윈 제어 연계



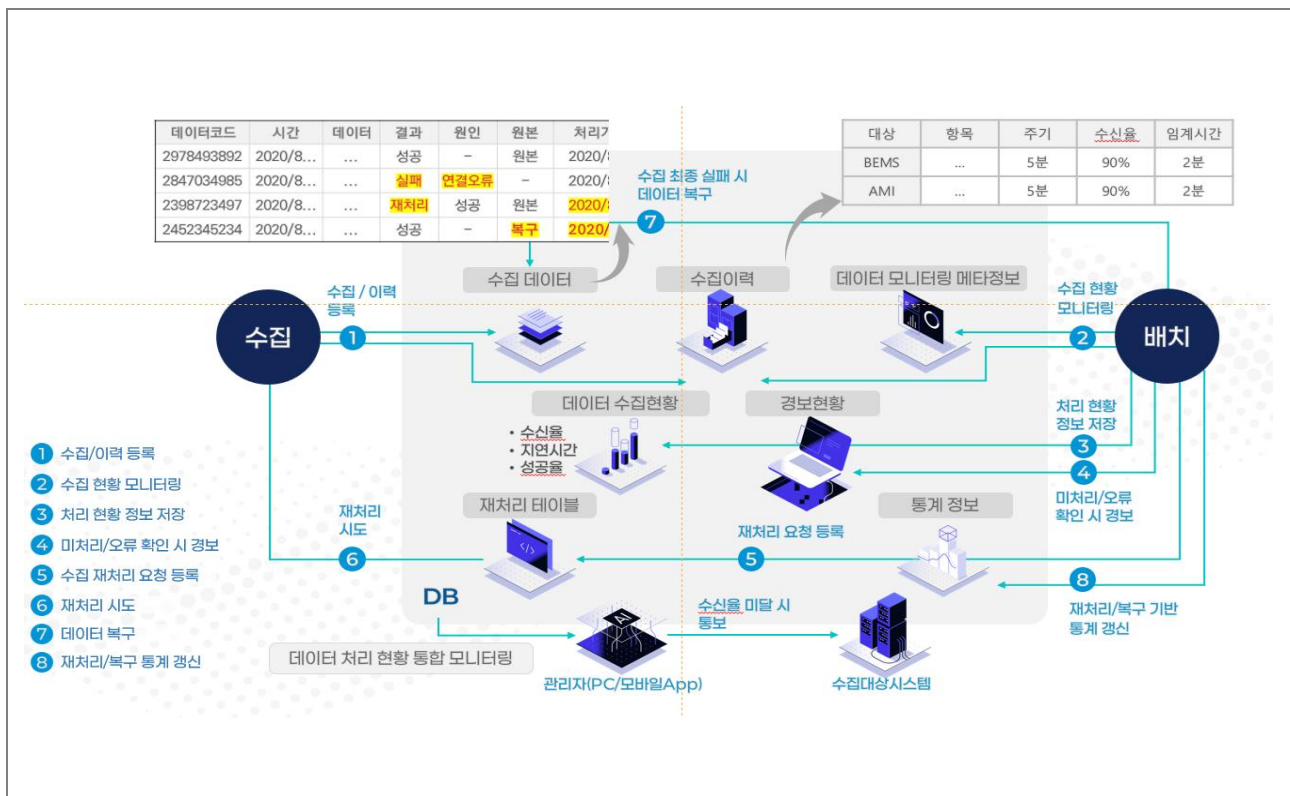
- IoT 디바이스 및 외부 시스템(기상청, KPX 등)으로부터 실시간 데이터를 수집하고, 메시지 큐 기반 비동기 처리로 안정적인 데이터 흐름을 보장하는 구조를 따르며, 수집 게이트웨이(GW-IoT, GW-S2S)를 통해 다양한 채널에서 들어오는 데이터를 일관되게 통합 관리하며, 데이터 누락이나 지연 없이 수집
- 수집된 데이터는 웹로그 및 재수집 여부를 기록하며, 수집 영역 DB 와 통계 영역 DB 로 이중 저장됨. 이후 배치엔진을 통해 5 분~1 개월 단위로 정제·보정되며, 이상값 자동 필터링과 무결성 검증을 통해 데이터 품질을 확보. 데이터 발생 시 자동으로 재수집 요청이 발생하고, 처리 이력은 통계 DB 에 연계됨.
- 고신뢰성 데이터 관리 구조를 기반으로, 디지털트윈 내 설비제어, 시뮬레이터 연동 등 다양한 기능과 연계됨. 수집-검증-활용까지 전주기 데이터 처리 체계를 통해 시스템 가용성과 연속성을 동시에 확보하며, 제어 정확도와 운영 자동화 수준을 향상시킬 수 있음.

□ 데이터 수집 구조: Collector-Queue-Loader 기반 분산 수집 파이프라인 적용



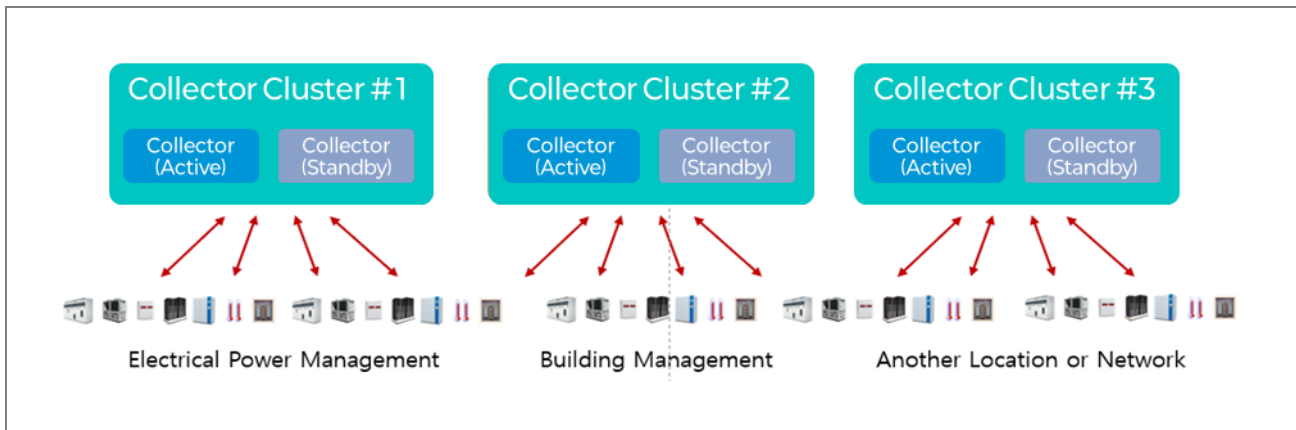
- 수많은 설비 및 IoT 센서에서 발생하는 데이터를 실시간으로 안정적으로 수집하고 처리하기 위해, Collector-Queue-Loader 구조의 분산형 파이프라인을 적용
- Collector 는 단일 또는 복수 노드로 구성되어 각 장치로부터 요청(Request)을 보내고 응답(Response)을 수신하며, 정제된 데이터를 메시지 큐 시스템(Distributed Queue)에 전달
- Queue 는 Topic 기반 분산 큐 시스템(Kafka 등)으로 구현되어, 데이터 유형별 구분 및 안정적인 버퍼링을 수행하고, 다수의 Loader는 해당 큐에 Subscribe 또는 Poll 방식으로 접근하여 데이터를 받아들이며, 대용량 데이터를 DB 로 Bulk Inserting 함.
- 수집-전달-저장의 모든 단계를 비동기·병렬 구조로 분산 처리함으로써, 데이터 유실 방지, 확장성 보장, 시스템 부하 분산 등의 효과를 제공하기 위함.

□ 서비스 품질 유지: 데이터 오류 복구 기반의 오류 감지 및 재수집 복구 처리 체계



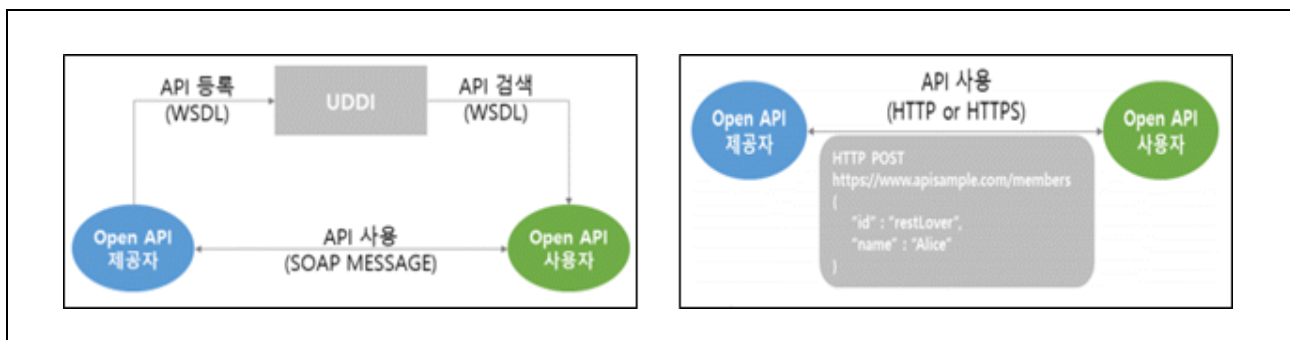
- 데이터 수집 이력 자동 기록 및 오류 감지를 기반으로 한 자율형 수집 복구 체계를 도입
- 각 수집 항목은 수집 성공 여부, 처리 지연 시간, 재처리 여부 등 메타정보와 함께 이력화 되며, 수집 실패 또는 지연 발생 시 관리자에게 실시간 경보가 전달됨.
- 수집 실패가 확인되면 자동으로 재처리 요청 등록 → 재수집 시도가 이루어지며, 재처리 결과 또한 이력으로 관리되며, 리 현황은 수집대상 시스템, 관리자 앱(Web/App) 등에 통합 제공됨
- 설정된 주기(예: 5 분)마다 각 항목별 수집률과 응답시간이 집계되어 SLA 준수 여부도 자동 검증되며, 수집 성공률, 복구율, 오류 유형별 통계를 기반으로 지속적인 품질 개선이 가능함.

□ **시스템 확장성:** 규모 증대 대응형 동적 확장 및 다중 클러스터링 기술 개발



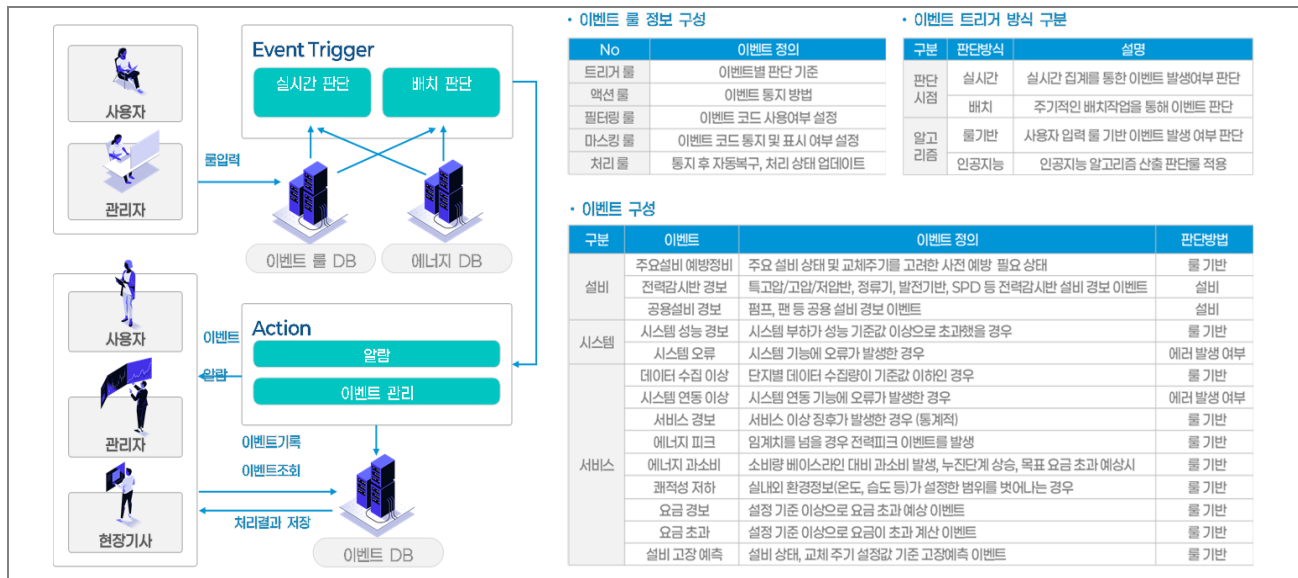
- 관제 대상 건물, 센서, 수집 노드의 지속적인 증가에 대응하기 위해 분산 처리 프로세싱 아키텍처를 적용하여, Collector-Queue-Loader 구조를 기본으로, 각 모듈은 독립적 서비스 단위(Microservice)로 운영되며, 노드 수평 확장(Scale-Out)이 가능한 구조로 설계
- 시스템은 모니터링 대상 증가 시 자동으로 수집-처리 인스턴스를 분산 배치하며, 이벤트 트리거, 실시간 분석, 경보 처리 등 고부하 기능 역시 분산 노드를 통해 병렬 실행하여, 시스템 전체 처리량과 응답속도 안정성이 지속 확보.
- 특히, 단일 건물 환경을 넘어 다중 건물, 지역 분산형 네트워크, 이기종 설비 연계와 같은 복잡한 물리 환경 변화에도 유연하게 대응할 수 있도록 클러스터 자동 구성 및 통합 관리 기능을 개발하여 운영 지속성과 확장성을 동시에 확보함.

□ **외부 연계 구조:** 개방형 API 연계 구조 설계



- 외부 시스템(기상, 설비, 공공 DB 등)과의 유연한 연계를 위해 개방형 API 기반 구조를 적용. EST 방식은 HTTP 기반 경량 통신(POST/GET 등)과 JSON 포맷을 활용하여 가볍고 확장성 높은 데이터 교환이 가능하며, 디지털트윈 플랫폼과의 실시간 연동, 사용자 맞춤형 서비스, 모바일 접근성 등 다양한 기능에 적용.
- RESTful API 구조를 기본으로 채택하되, 공공기관 등 SOAP 방식 연계가 필요한 경우를 대비해 이중 인터페이스 구조(REST + SOAP 호환성)를 함께 제공하여 시스템 개방성과 범용성을 동시에 확보할 계획

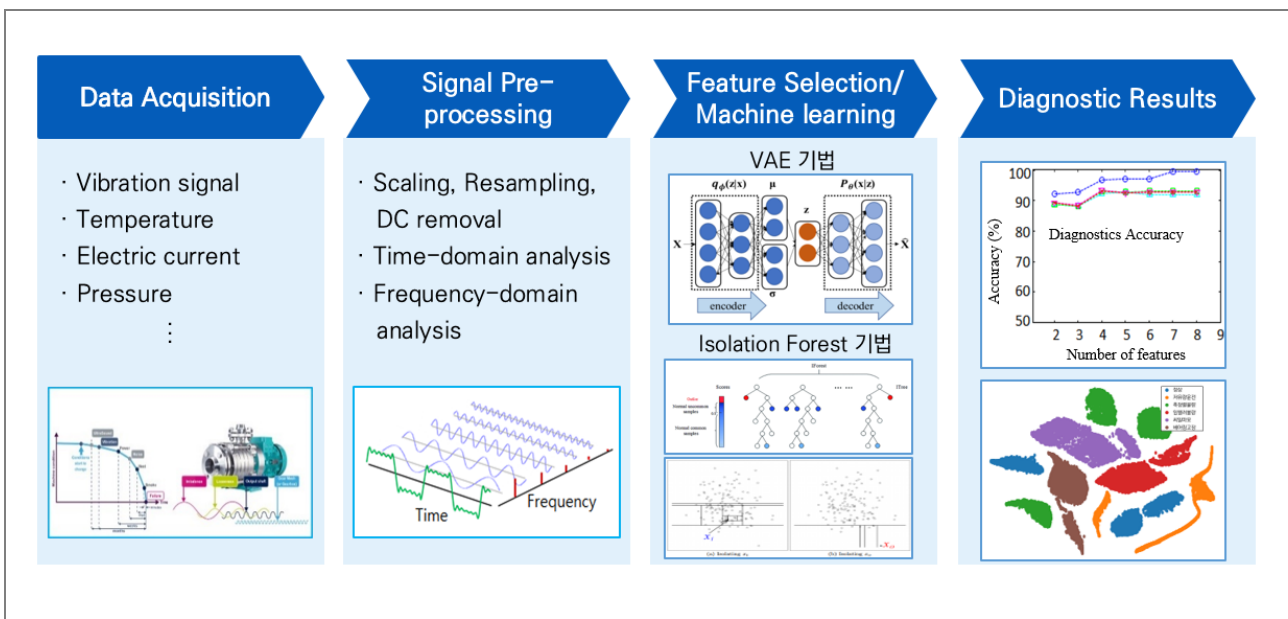
□ 실시간 운영대응: 이벤트 기반 알림 및 자동 조치 시스템



- 설비 및 공간 운영 과정에서 발생하는 이상 현상을 이벤트로 정의하고, 이를 기반으로 자동 알림 및 조치가 가능한 구조를 구축
- 이벤트 발생은 실시간 판단(센서/시뮬레이션) 또는 배치 판단(정기 점검, 이력 비교 등)을 통해 감지되며, 트리거 조건은 유형별(시간, 횟수, 임계값, 인공지능 기반 등)로 세분화
- 각 이벤트는 '설비 이상', '시스템 오류', '공간 환경 초과', '서비스 장애' 등으로 구분되며, 발생 시 이벤트 DB 에 등록되고 관리자는 물론 사용자, 현장 기사에게 역할별로 최적화된 알림과 대응 지시가 자동 전달되며, 관리자 인터페이스를 통해 이벤트 등록 기준, 필터링 조건, 알림 채널 설정 등도 유연하게 구성 가능
- 행동 기반 자동조치(Action Rule)로 발전할 수 있는 기반이며, 향후 Co-pilot 시스템이나 M/L 기반 의사결정 추천 기능과도 연계 가능
- 공조설비 최적제어 및 학습 피드백 루프 설계 전략
 - 고층빌딩 환경 내 공조설비의 에너지 효율과 쾌적도를 동시에 최적화하기 위해, 디지털트윈 플랫폼과 MLOps 기반 예측 모델을 연동한 M/L 피드백 루프 구조를 설계
 - 디지털트윈 플랫폼은 고정밀도로 수집된 실내외 환경, 설비 상태, 공간 활용도 데이터를 기반으로 공조설비의 운전 결과와 환경 영향을 실시간 추적하고, 이 데이터는 Feature Engineering 과정을 거쳐 정제되며, Regression, Tree, Neural Network 등 다양한 알고리즘을 통해 모델을 학습하고 평가함.
 - 학습된 모델은 실시간 또는 배치 방식으로 운영 플랫폼에 배포되어, 설비 운전 조건별 최적 제어값을 예측하여, 이후 제어 결과는 다시 데이터로 수집되어 반복적 학습(MLOps 루프)이 가능하며, 이를 통해 지속적으로 공조설비 제어의 정확도·에너지 절감율·쾌적도 유지율을 향상시킴.



- 예측 유지보수 적용 알고리즘



- 설비의 이상 징후를 사전에 감지하고 고장을 예방하기 위해, 진동, 전류, 압력, 온도 등의 실시간 신호 데이터를 분석하는 예측 유지보수 알고리즘을 도입(데이터 연동에 따라 정확도는 차이 발생)
- 센서 기반으로 설비 진동, 온도, 전류, 압력 등의 원천 데이터를 수집(Data Acquisition)하며, 시간영역 및 주파수 영역 분석, 스케일링, DC 제거, 리샘플링 등 신호 전처리(Signal Pre-processing)를 수행
- VAE(Variational Autoencoder), Isolation Forest 등의 비지도 학습 기반 알고리즘을 활용하여 이상 탐지 및 고장 분류를 수행하고, 정확도 기반 피쳐 조합 평가, 클러스터링 기반 진단 결과 시각화를 통해 설비 이상 유형별 분류 결과를 제공할 수 있음.

2.3 수행 일정

업무 및 개발 내용		'25년												'26년						
		5	6	7	8	9	10	11	12	1	2	3	4	5	6	7				
과제 용역 계약																				
요구사항 정의 및 분석																				
빌딩 설비 연동 제어 기술 개발	- 설비 연동																			
	- 설비 제어																			
	- 실시간 모니터링 및 알림																			
	- 플랫폼 탑재/Test																			
Asset 자동 동기화 기술 개발	- 자재 DB 구축																			
	- 수명 및 이력 관리																			
	- 자재 상태 모니터링																			
	- 플랫폼 탑재/Test																			
3D 모델 자동 동기화 기술 개발	- 건물 3D 모델링 자동화																			
	- 구조 요소 자동 인식																			
	- 건물 변화 추적																			
	- BIM 정보 통합																			
	- 플랫폼 탑재/Test																			
통합 시뮬레이션 기술 개발	- 시뮬레이터 인터페이스																			
	- 통합 시뮬레이션 환경 구축																			
BEES 플랫폼 고도화	- 공간 최적 운영																			
	- 에너지 통합 관리																			
	- 통합 재난 대응																			
	- 보안 안전 관리																			
	- 플랫폼 탑재/Test																			
종합 시스템 테스트 및 시범 운영																				

□ 단계별 세부 마일스톤 및 일정계획

개발단계	기간 (개월)	주요 세부 업무 및 산출물	마일스톤
요구사항 정의 및 분석	2	- 이해관계자 인터뷰, 워크숍 진행 - 서비스별 기능/시나리오 도출 - 공동 인터페이스 정의 - 요구사항 명세서 작성	M1: 요구사항 명세서 승인 (2 개월차 말)
시스템 아키텍처 및 설계	2	- 전체 플랫폼 및 마이크로서비스 아키텍처 설계 - 서비스별 상세 설계서 (기능, 데이터, API) - UI/UX 와이어프레임, 프로토타입 - 보안 정책 확정	M2: 설계서 완성 (3 개월차 말)
개발 및 단위 테스트	3~12	- 각 서비스별 모듈 개발 시작 - 관제: 설비 연동·제어, 실시간 모니터링 구현 - 자산관리: 자재 DB, CRUD, 예측 알고리즘 - 유지보수: 3D 모델링 파이프라인, 구조 인식 - 운영관리: 에너지 시뮬레이션, 재난/대피, 보안/통제, 탄소관리 - 유닛 테스트, 스프린트 리뷰, CI 구축	M3: 1 차 주요 모듈 개발 완료 (11 개월차) M4: 전체 개발 완료 (13 개월차)
통합 및 연동 테스트	13	- 각 서비스 모듈 통합 빌드 - 인터페이스 연계 및 API 통합 테스트 - 데이터 흐름, 보안, 메시지 전달 점검 - 모듈 간 오류 수정 및 회귀 테스트	M5: 통합 빌드 및 연동 테스트 완료 (14 개월차)
종합 시스템 테스트 및 품질 검증	13	- 기능, 시스템, 부하/성능, 보안, UAT 테스트 - 자동화 및 수동 테스트 수행 - 최종 회귀 테스트 및 품질 인증 산출물 확보	M6: 종합 테스트 완료 (14 개월차)
시범 운영 및 최종 배포 준비	15	- 제한적 시범 운영(Pilot) 실시 및 피드백 반영 - 운영 환경 구축, 데이터 이행, 백업/복구 점검 - 운영 인수인계, 배포 리허설, 최종 점검	M7: 시범 운영 완료 (15 개월차) M8: 최종 일괄 배포 (15 개월차)

